

Premier University
Department Of Computer Science and Engineering



Final Project Report on
Asteroids Style Space Shooter

Course Name: Computer Graphics and Image Processing Laboratory
Course Code : CSE 456
Section : B
Semester : 7 th
Submission Date: 02-03-2025.

Submitted by:

Ovi Das ID: 2104010202187 (35%)
Refatul Islam ID: 2104010202188 (25%)
Binoy Chakraborty ID: 2104010202207 (20%)
Brishty Saha ID: 2104010202218 (20%)

Under the Supervision of

Salman Farsi

Lecturer

Department Of Computer Science and Engineering
Premier University, Chittagong

Contents

1	Introduction	3
1.1	Introduction	3
2	Objective	3
2.1	Objective	3
3	Motivation	4
3.1	Motivation	4
4	Requirement Specification	5
4.1	Requirement Specification	5
5	Methodology And Feature Description	6
5.1	Methodology And Feature Description	6
6	Implementation and Pseudocode	8
6.1	Implementation	8
6.1.1	Round 1 Gameplay Screen	9
6.1.2	Gameplay Screen (Round 2)	10
6.1.3	Gameplay Screen (Round 3)	11
6.1.4	Gameplay Screen	12
6.1.5	Final Game Over Screen (Detailed)	13
6.1.6	: After Game Over & Restarting the Game	14
6.2	Pseudocode	15
7	Application	17
7.1	Application	17
8	Future Scope	18
8.1	Future Scope	18
9	Conclusion	19
9.1	Conclusion	19

1 Introduction

1.1 Introduction

The Asteroids-Style Space Shooter is a 2D arcade-style game that immerses players in an action-packed space adventure. The game challenges players to navigate a spaceship through an asteroid field while avoiding collisions and destroying asteroids. Inspired by the classic arcade game Asteroids, this modern adaptation features improved graphics, engaging sound effects, and enhanced gameplay mechanics.

Players control a spaceship with smooth and responsive mechanics, maneuvering through an ever-increasing wave of asteroids. The game introduces various asteroid types, each with unique movement patterns and behaviors. Additionally, a dynamic scoring system encourages high-score achievements while increasing difficulty over time.

With an emphasis on fast-paced gameplay, the game rewards skillful dodging and precise shooting. Explosions, particle effects, and immersive background music enhance the gaming experience. The ultimate goal is to survive as long as possible, rack up points, and compete for the highest score.[1]

2 Objective

2.1 Objective

The necessary objectives of our project are given:

1. Develop a playable and engaging space shooter game.
2. Implement smooth player controls and physics.
3. Create multiple asteroid types and movement patterns.
4. Include an interactive scoring system and difficulty progression.
5. Enhance gameplay with sound effects and animations. [1]

3 Motivation

3.1 Motivation

Creating an Asteroids-style space shooter requires motivation to drive design and development.

1. Classic Arcade Nostalgia

Many developers and players have a deep appreciation for retro arcade games. An Asteroids-style shooter captures the fast-paced, simple yet challenging mechanics that made classic games so beloved. Reviving this style appeals to both older gamers and new audiences looking for a fun and accessible experience.

2. Skill-Based Gameplay

Unlike modern games that rely on complex narratives or RPG elements, space shooters focus on player skill—dodging asteroids, precise shooting, and quick reflexes. This creates an engaging challenge where improvement is noticeable, keeping players motivated to get better with each playthrough.

3. Endless Replayability

Procedurally generated asteroid patterns, increasing difficulty, and score-based progression make the game endlessly replayable. Since each game session is unique, players are encouraged to return and beat their high scores, fostering long-term engagement.

4. Simple Yet Expandable Design

The basic mechanics of an Asteroids-style game (movement, shooting, and dodging) are easy to implement, making it an ideal project for indie developers or beginners. At the same time, it offers room for expansion—power-ups, enemy ships, multiplayer modes, or even roguelike elements can enhance the experience without breaking the core gameplay loop.[2]

4 Requirement Specification

4.1 Requirement Specification

Hardware Requirements:

- i) A computer with a modern CPU and GPU (integrated or dedicated).
- ii) Minimum 4GB RAM (more recommended).
- iii) Keyboard and mouse for input.

Software Requirements:

- 1) **Operating System:** Windows, Code Blocks(20.3).
- 2) **Development Environment:** Any C++ compiler (MinGW).
- 3) **Libraries:**
 - OpenGL for graphics rendering.
 - GLUT (OpenGL Utility Toolkit) for windowing and input handling.
 - GLU (OpenGL Utility Library) for 2D and 3D rendering utilities.

5 Methodology And Feature Description

5.1 Methodology And Feature Description

Our Asteroids Style Space Shooter project methodology is given below:

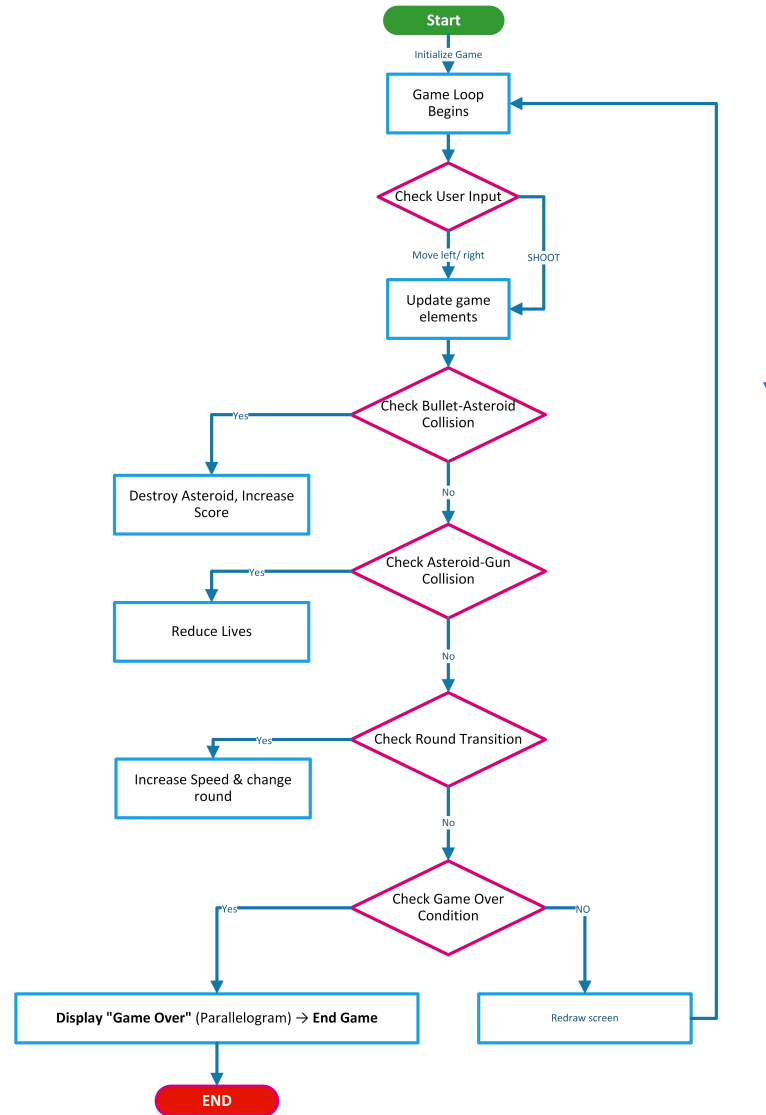


Fig: Methodology

The flowchart shows the basic steps of a game loop for a simple game, likely involving a player-controlled gun shooting at asteroids. Here's a step-by-step explanation of the process:

1. **Start:** The game begins.
2. **Initialize Game:** The game sets up initial conditions, such as player position, asteroid positions, score, lives, and other game elements.
3. **Game Loop Begins:** The main game loop starts, which will continuously run until the game ends.
4. **Check User Input:** The game checks for player input, such as moving left or right and shooting.
5. **Move Left/Right:** Based on the input, the player's gun moves accordingly.
6. **SHOOT:** If the player shoots, a bullet is fired.

7. Update the elements of the game: The game updates the positions of all elements, such as bullets, asteroids, and the player's gun.

8. Check Bullet-Asteroid Collision: The game checks if any bullet has collided with an asteroid.

Yes: If a collision is detected, the asteroid is destroyed, and the player's score increases.

No: If no collision is detected, the game proceeds to the next check.

9. Check Asteroid-Gun Collision: The game checks if any asteroid has collided with the player's gun.

Yes: If a collision is detected, the player loses a life.

No: If no collision is detected, the game proceeds to the next step.

10. Increase Speed and Change Round: The game may increase the speed of the asteroids and change the round, making the game more challenging.

11. Check Round Transition: The game checks if the conditions for transitioning to the next round are met.

Yes: If the conditions are met, the game transitions to the next round.

No: If not, the game proceeds to the next check.

12. Check Game Over Condition: The game checks if the game over condition is met (e.g., player lives are zero).

NO: If the game is not over, the loop continues, and the screen is redrawn.

YES: If the game is over, the game displays "Game Over" and ends.

13. Display "Game Over": The game displays a "Game Over" message, typically in a parallelogram shape, indicating the end of the game.

14. End Game: The game ends, and the loop stops.

15. Redraw Screen: If the game is not over, the screen is redrawn with updated positions of all elements, and the loop continues.

This loop continues until the game over condition is met, at which point the game ends.

6 Implementation and Pseudocode

6.1 Implementation

This implementation include :

Testing and Debugging – Identifying and fixing errors to ensure correct functionality.

Integration and Deployment – Merging components and deploying the system for use.

Game Overview That's a 2D asteroid-shooting game running in a window.

Game Interface

- **Main Window:** Displays a black screen with a green triangle representing the player's spaceship. The text "Round 1" appears in yellow.
- **UI Elements:**
 - **Score:** Displays the current score (starting at 0).
 - **Lives:** Shows the remaining lives (default: 3).
 - **Controls:** Three buttons – "Pause/Resume," "Restart Game," and "End Game."

Additional Details

- The game is in its early development stage with minimalistic visuals.
- A second, partially visible black window suggests a debugging console or another application running along.

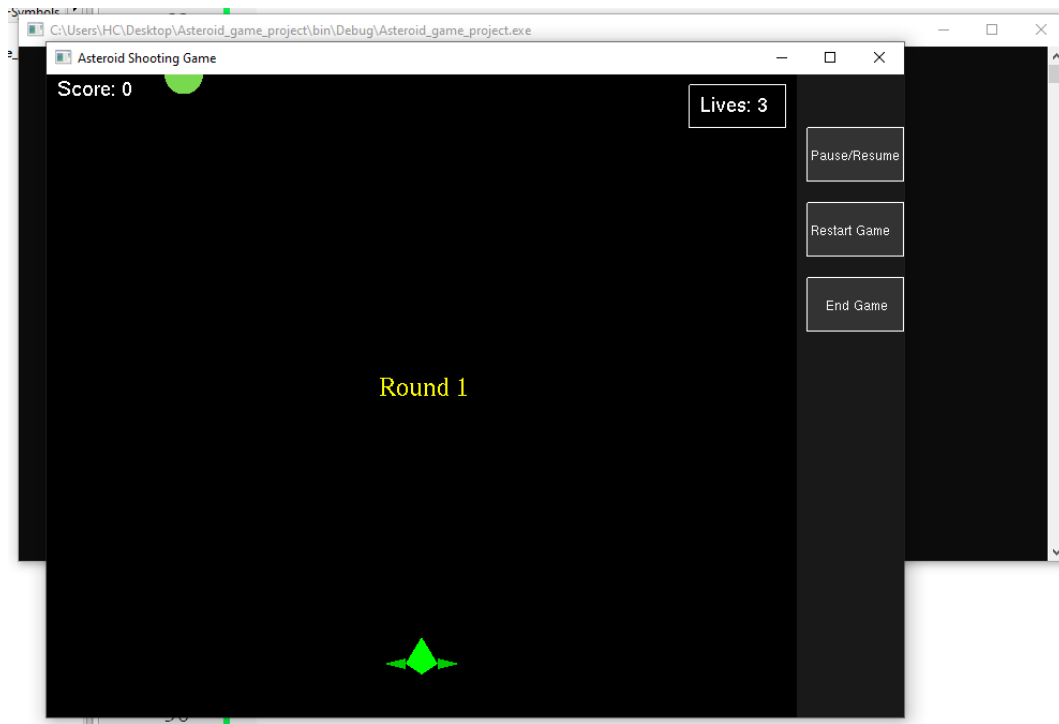


Fig: 01

6.1.1 Round 1 Gameplay Screen

- **What it shows:**

- The game is in **round 1** (the first level of the game).
- The player's **Score** is **0** (starting fresh).
- The player has **3 lives** remaining (default starting lives).
- The player is actively shooting asteroids or avoiding obstacles.
- Buttons like **Pause/Resume**, **Restart Game**, and **End Game** are visible.

- **Explanation:**

This screenshot shows the game during **Round 1**, the first level of the game. The player has just started, so the score is **0**, and they have **3 Lives** to survive. The player is actively engaged in shooting asteroids or avoiding obstacles to progress in the game. The buttons for controlling the game (pause, restart, or end) are also displayed, allowing the player to manage the gameplay.

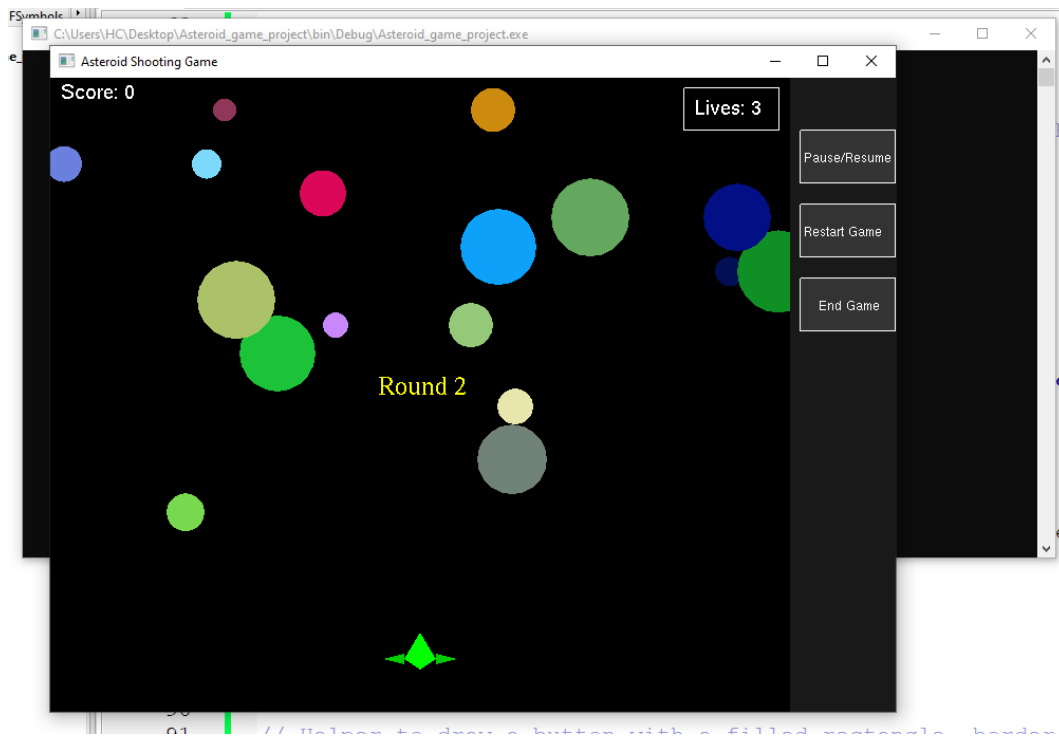


Fig:02

6.1.2 Gameplay Screen (Round 2)

- **What it shows:**
 - The game has progressed to **Round 2**.
 - The player's score has increased to **150**.
 - The player still has **3 Lives**.
- **Explanation:**

The player has successfully completed Round 1 and moved to Round 2. The score has increased, indicating progress.

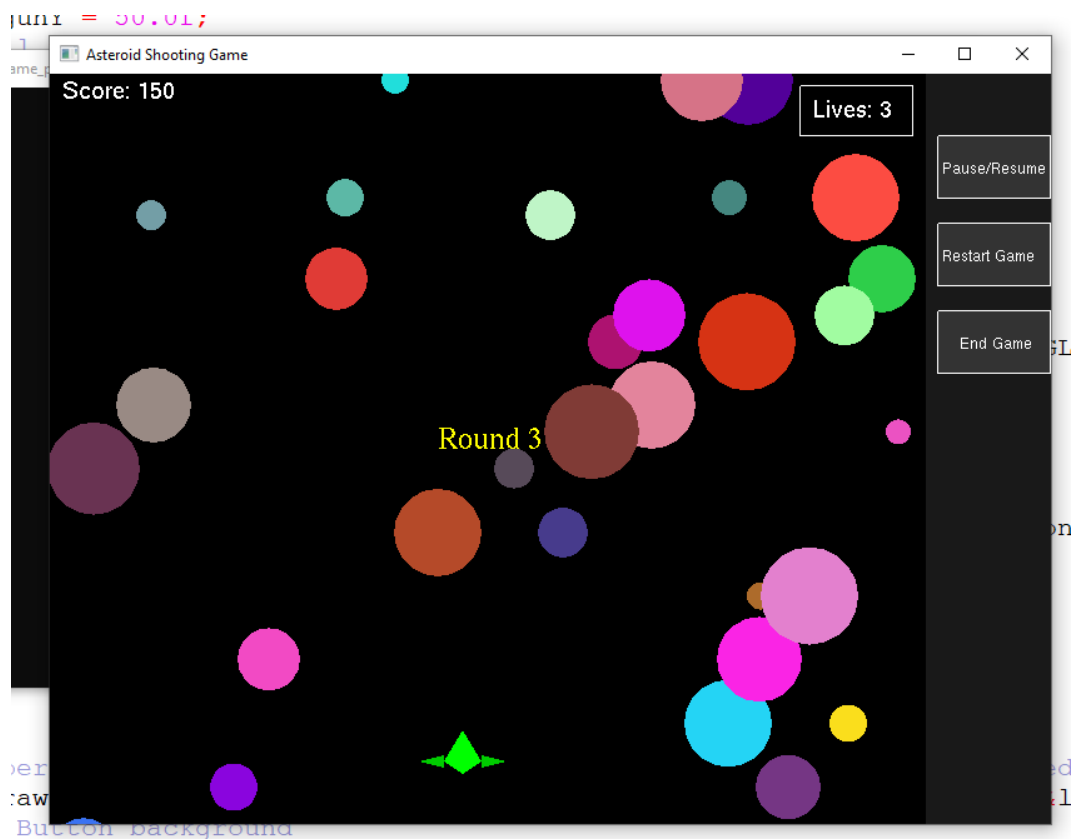


Fig:03

6.1.3 Gameplay Screen (Round 3)

- **What it shows:**

- The is in **round 3**.
- The player's game score has been increased to **150**.
- The player now has only **3 Life** left.

- **Explanation:**

The player is in a more advanced round, with a higher score but fewer lives. The game is becoming more challenging by increasing astroids speed.

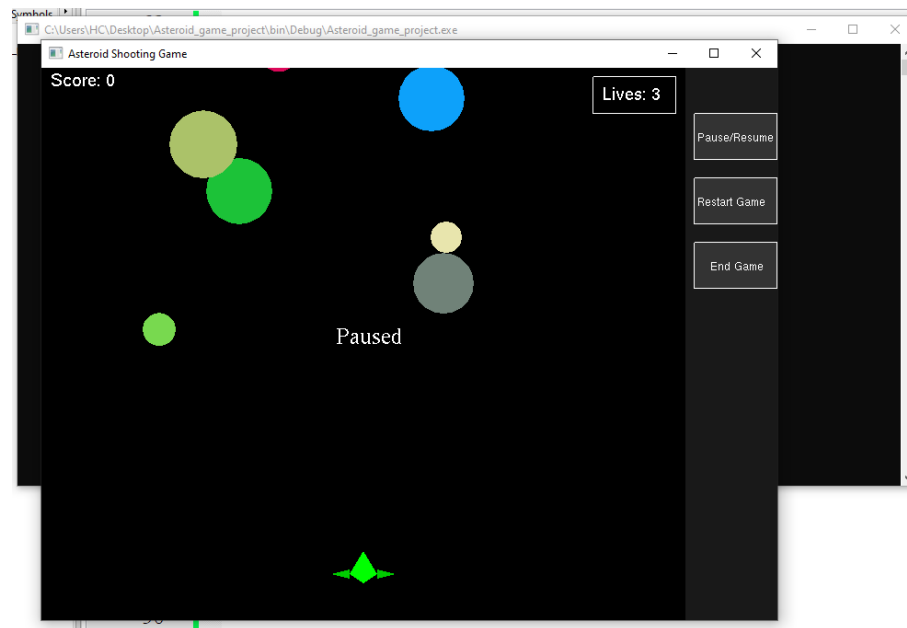


Fig:04

Pause Screen:

- The game can be paused, and the screen shows options like **Pause/Resume**, **Restart Game**, and **End Game**.
- When paused, the gameplay is temporarily stopped, and the player can choose to continue or restart.

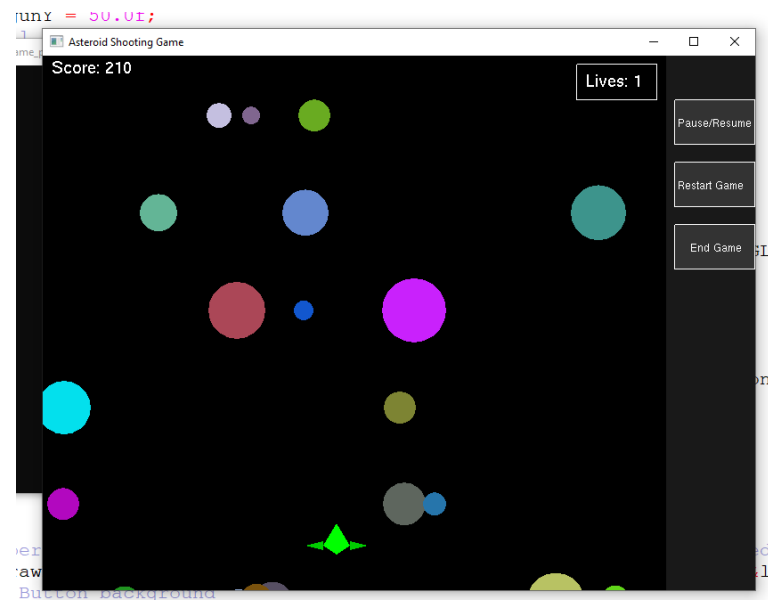


Fig:05

6.1.4 Gameplay Screen

- What it shows:
 - same as round 3
 - The player still has **1 Life** remaining score increased to 210.

- **Explanation:** The player is in a more advanced round, with a higher score but still only **1 Life**. The game is becoming more challenging as the player progresses.

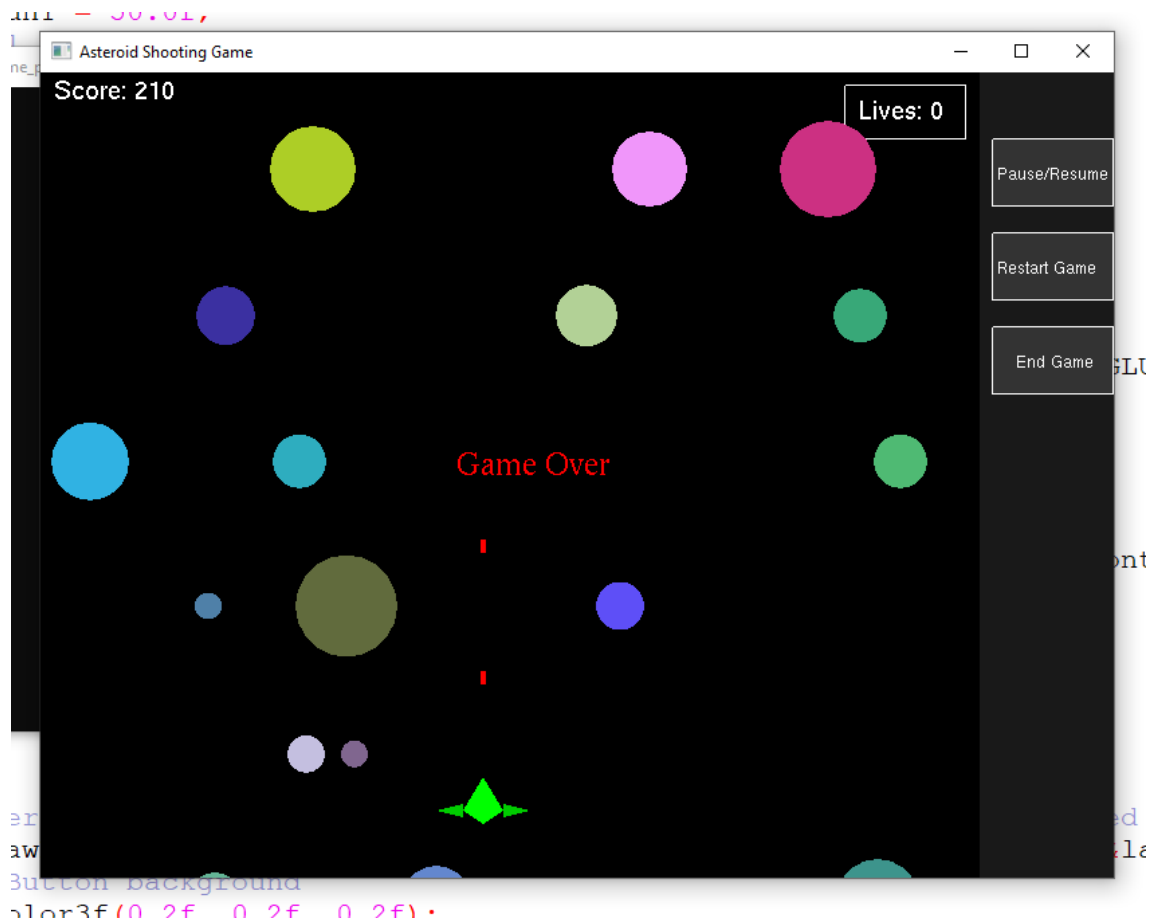


Fig:06

6.1.5 Final Game Over Screen (Detailed)

- **What it shows:**
 - A more detailed **Game Over** screen with numbers and symbols after collision.
 - The final score is **210**.
- **Explanation:**

This is a more detailed view of the game over screen, after collision with asteroids and end of lives.

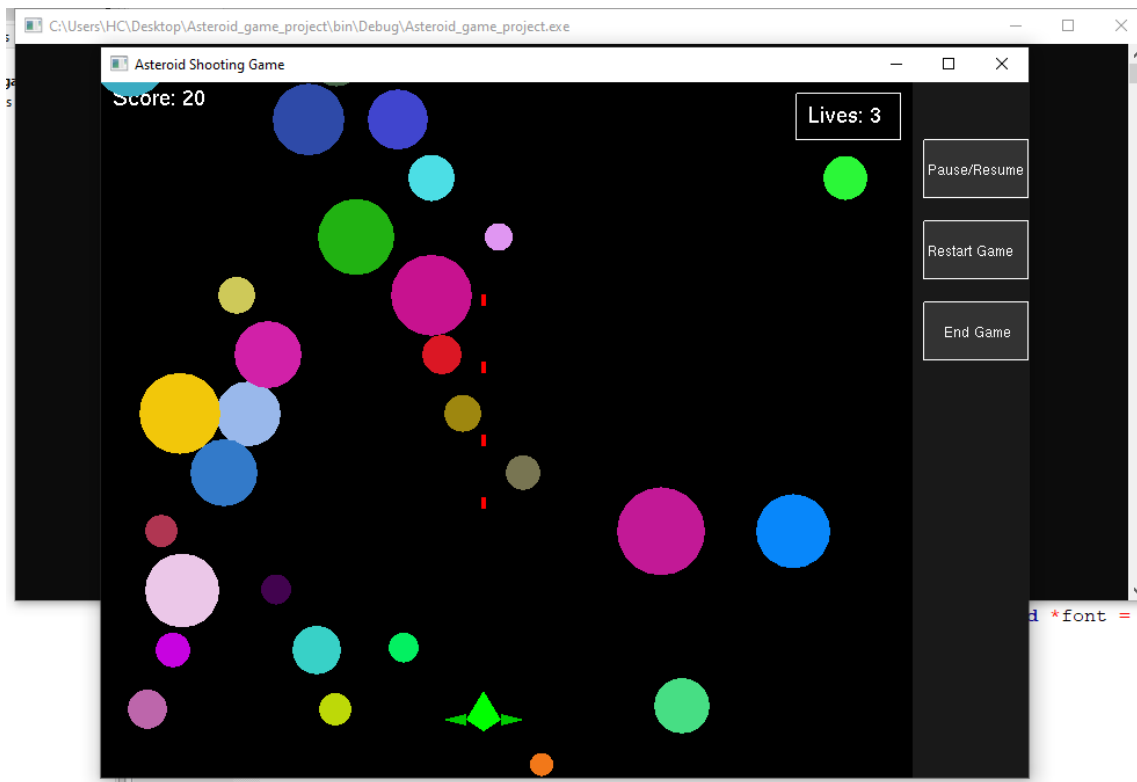


Fig:07

6.1.6 : After Game Over & Restarting the Game

- **What it shows:**

- The game has been **restarted** after a previous game over.
- The player has been playing for some time, and the **Score** is now displayed (**20**).
- The player has **3 Lives** remaining. AS it is a new round of game.

- **Explanation:**

This screenshot shows the game after the player has restarted it following a previous game over. The player has been playing for a while, and the score has increased to **20**. However, the player still has **3 Life** to survive, making the game bit challenging. This screen reflects the progress made after restarting the game.

6.2 Pseudocode

```
start:
IF gameOver THEN
    DISPLAY "Game Over"
    IF restart button pressed THEN
        RESET game state (score, lives, round, positions, clear objects)
    ENDIF
ELSE
    IF paused THEN
        DISPLAY "Paused"
    ELSE
        UPDATE timer and compute delta time (dt)
        IF round duration elapsed THEN
            TRANSITION to next round (adjust asteroid speed and spawn rate)
        ENDIF
        IF spawn interval elapsed THEN
            SPAWN new asteroids (number based on current round)
        ENDIF
        FOR EACH asteroid DO
            UPDATE asteroid position
            IF asteroid collides with player THEN
                DECREASE lives; IF lives <= 0 THEN SET gameOver
            ELSE IF asteroid off-screen THEN
                REMOVE asteroid
            ENDIF
        ENDFOR
        FOR EACH bullet DO
            UPDATE bullet position
            IF bullet off-screen THEN
                DEACTIVATE bullet
            ENDIF
            FOR EACH asteroid DO
                IF bullet collides with asteroid THEN
                    INCREASE score; REMOVE asteroid; DEACTIVATE bullet; BREAK loop
                ENDIF
            ENDFOR
        ENDFOR
        RENDER game area, HUD, asteroids, bullets, player's plane, and sidebar
    ENDIF
    PROCESS user inputs (keyboard for shooting/movement, mouse for buttons)
ENDIF
GO TO :start: and repeat
END
```

This pseudocode describes the game loop for an asteroid-shooting game. Let's break it down step by step:

Start of the Game Loop: The game loop starts and repeatedly checks conditions to control the game flow.

Handling Game Over State: If `gameOver` is true, the message "Game Over" is displayed. If the player presses the restart button, the game state is reset (score, lives, round, positions, and clearing objects).

Handling Paused State: If the game is paused, the "Paused" message is displayed, and the loop waits.

Updating the Game (When Not Paused or Over): The timer is updated, and `dt` (delta time) is computed. If the round duration has elapsed, the game transitions to the next round (adjusting asteroid speed and spawn rate). If the spawn interval has elapsed, new asteroids are spawned.

Processing Asteroid Behavior: Each asteroid's position is updated. If an asteroid collides with the player, the player's lives decrease. If lives reach zero, `gameOver` is set to true. If an asteroid moves off-screen, it is removed.

Processing Bullet Behavior: Each bullet's position is updated. If a bullet moves off-screen, it is deactivated. Each bullet is checked for collision with asteroids: If a bullet collides with an asteroid, the score increases, the asteroid is removed, and the bullet is deactivated. The loop is exited early after a collision (to prevent unnecessary checks).

Rendering the Game:

The game area, HUD (Heads-Up Display), asteroids, bullets, player's plane, and sidebar are drawn.

Processing User Inputs Keyboard inputs are handled for movement and shooting. Mouse inputs are processed for buttons.

Looping the Game: The game loop returns to the Start state and repeats until the game is over or paused.

End of the Game Loop The game continues running until the game-over condition is met, after which it can be restarted.[3]

7 Application

7.1 Application

An Asteroids-style space shooter has several applications in making it a great project for experimenting with various techniques. Here are some key applications:

1. Real-Time Rendering and 2D Graphics Techniques

- The game involves rendering objects like spaceships, asteroids, and projectiles in real time, utilizing 2D transformations such as scaling, rotation, and translation.
- It can use sprite-based rendering, shaders, and lighting effects to enhance visual appeal.

2. Procedural Generation

- Asteroid shapes, backgrounds (stars, planets), and even enemy waves can be procedurally generated using algorithms like Perlin noise or fractals.
- This ensures variety in gameplay while minimizing storage for assets.

3. Collision Detection and Image Processing

- Techniques like bounding box, circle collision detection, or pixel-perfect collision detection can be implemented for interactions between the spaceship, asteroids, and bullets.
- Motion blur, explosion effects, and impact visuals can be enhanced using image processing techniques.

4. Particle Systems for Visual Effects

- Explosions, thruster flames, and asteroid destruction can be implemented using particle systems, which simulate the movement of small particles to create realistic effects.
- Can use GPU-accelerated particles for smoother performance.

5. Texture Mapping and Image Filtering

- Space backgrounds and asteroid surfaces can use texture mapping with filtering techniques (bilinear/trilinear filtering, mipmapping) to improve visual quality.
- Post-processing effects like bloom, glow, or motion blur can be added for a modern look. [3]

8 Future Scope

8.1 Future Scope

As the game develops, several enhancements and expansions can be made to improve gameplay, engagement, and overall user experience. Here are some possible future improvements:

1. Gameplay Enhancements

Power-ups and Upgrades: Introduce temporary power-ups such as shields, rapid fire, or extra lives.

New Enemy Types: Introduce different asteroid types (e.g., splitting asteroids, faster asteroids, explosive ones). **Boss Battles:** Add periodic boss fights to increase challenge and variety.

Dynamic Environments: Introduce environmental effects like space debris, black holes, or moving obstacles.

2. Graphics and UI Improvements

Improved Visual Effects: Add particle effects, explosions, and lighting for a better visual experience.

Animated UI Elements: Improve HUD and menu animations to make interactions smoother.

Customization Options: Allow players to change spaceship designs, colors, and effects.

3. Multiplayer and Social Features

Online Multiplayer Mode: Introduce co-op or competitive modes where players can battle against each other. **Leaderboards and Achievements:** Implement a ranking system and unlockable achievements for better engagement.

Friend System and Challenges: Allow players to challenge friends or compare scores.

4. Additional Game Modes

Survival Mode: An endless wave-based mode with increasing difficulty.

Time Attack Mode: Players must achieve the highest score within a time limit.

Campaign Mode: A story-driven mode with different missions and objectives.

5. Cross-Platform Availability

Mobile Support: Develop a mobile version with touch controls.

VR/AR Support: Explore immersive gameplay with VR or AR capabilities.

Cross-Platform Play: Allow seamless gameplay between PC, consoles, and mobile devices. [4]

9 Conclusion

9.1 Conclusion

The Asteroids-style space shooter game is an engaging 2D experience where the player controls a plane to shoot and destroy incoming asteroids while avoiding collisions. The game includes multiple rounds with increasing difficulty, a score tracker, and a limited number of lives. A sidebar provides essential game controls like pausing, restarting, or exiting. The game's mechanics, including asteroid movement, bullet firing, and collision detection, are well-implemented, offering a solid foundation. With further improvements such as sound effects, dynamic difficulty scaling, and power-ups, the game can provide a more immersive and challenging experience for players.[2]

References

- [1] D. Singh, E. J. Leavline, and B. S. Baig, “Diabetes prediction using medical data,” *Journal of Computational Intelligence in Bioinformatics*, vol. 10, no. 1, pp. 1–8, 2017.
- [2] Z. Tafa, N. Pervetica, and B. Karahoda, “An intelligent system for diabetes prediction,” in *2015 4th Mediterranean Conference on Embedded Computing (MECO)*, pp. 378–382, IEEE, 2015.
- [3] D. Sisodia and D. S. Sisodia, “Prediction of diabetes using classification algorithms,” *Procedia computer science*, vol. 132, pp. 1578–1585, 2018.
- [4] Y. Chang, L. Huang, S. Shin, K. Lin, K. Chong, F. Yen, H. Chang, S. Chuang, T. Hsieh, C. Hsiung, *et al.*, “A point-based mortality prediction system for older adults with diabetes,” *Scientific Reports*, vol. 7, no. 1, p. 12652, 2017.